

2ourc3's 1 - Baby Crackme

vendredi 29 mai 2026 00:26

```
; Attributes: bp-based frame

; int __fastcall main(int argc, const char **argv, const char **envp)
public main
main proc near

var= dword ptr -4

push    rbp
mov     rbp, rsp
sub     rsp, 30h
call    __main
mov     [rbp+var], 0
lea     rcx, Format          ; "Enter the Key to decrypt the code: "
call    printf
lea     rax, [rbp+var]
mov     rdx, rax
lea     rcx, aD              ; "%d"
call    scanf
mov     eax, [rbp+var]
mov     ecx, eax             ; var
call    FUN
mov     eax, 0
add     rsp, 30h
pop     rbp
retn
main endp
```

```
; Attributes: bp-based frame

; char __cdecl FUN(int var)
public FUN
FUN proc near

vars= byte ptr -30h
var2= dword ptr -8
i= dword ptr -4
var= dword ptr 10h

push    rbp
mov     rbp, rsp
sub     rsp, 50h
mov     [rbp+var], ecx
mov     qword ptr [rbp+vars], 0
mov     qword ptr [rbp+vars+8], 0
mov     qword ptr [rbp+vars+10h], 0
mov     [rbp+vars+18h], 0
mov     [rbp+vars+1], 22h ; 'x'
mov     [rbp+vars+1], 33h ; '3'
mov     [rbp+vars+2], 20h ; ' '
mov     [rbp+vars+3], 22h ; 'x'
mov     [rbp+vars+4], 2Ah ; 'A'
mov     [rbp+vars+5], 2Ch ; ','
mov     [rbp+vars+6], 24h ; '$'
lea     rax, [rbp+vars]
mov     rcx, rax             ; Str
call    strlen
mov     [rbp+var2], eax
mov     [rbp+i], 0
jmp     short loc_40150B
```

```
loc_40150B:
mov     eax, [rbp+i]
cmp     eax, [rbp+var2]
jnl     short loc_4015AB
```

```
loc_40150B:
mov     eax, [rbp+i]
cmp     eax, [rbp+var2]
jnl     short loc_4015AB
```

```
loc_4015AB:
mov     eax, [rbp+i]
cdqe
movzx   eax, [rbp+rax+vars]
mov     edx, [rbp+var]
xor     edx, eax
mov     eax, [rbp+i]
cdqe
mov     [rbp+rax+vars], dl
mov     eax, [rbp+i]
cdqe
movzx   eax, [rbp+rax+vars]
movsx   eax, al
mov     ecx, eax             ; Character
call    putchar
add     [rbp+i], 1
```

```
nop
add     rsp, 50h
pop     rbp
retn
FUN endp
```

```
; Attributes: bp-based frame

; char __cdecl FUN(int var)
public FUN
FUN proc near

vars= byte ptr -30h
var2= dword ptr -8
i= dword ptr -4
```

```

var= dword ptr 10h

push    rbp
mov     rbp, rsp
sub     rsp, 50h
mov     [rbp+var], ecx
mov     qword ptr [rbp+vars], 0
mov     qword ptr [rbp+vars+8], 0
mov     qword ptr [rbp+vars+10h], 0
mov     [rbp+vars+18h], 0 ; => on met 0 à l'adresse +18h
mov     [rbp+vars], 22h ; '=' ; => on met " au base adresse de vars (vars[0])??
mov     [rbp+vars+1], 33h ; '3' ; => on met 3 à vars (vars[1])
mov     [rbp+vars+2], 20h ; ' ' ; => on met espace à vars (vars[2])
mov     [rbp+vars+3], 22h ; '=' ; => on met " à vars (vars[3])
mov     [rbp+vars+4], 2Ah ; '*' ; => on met * à vars (vars[4])
mov     [rbp+vars+5], 2Ch ; ',' ; => on met , à vars (vars[5])
mov     [rbp+vars+6], 24h ; '$' ; => on met $ à vars (vars[6])
lea     rax, [rbp+vars] ; => on charge l'adresse du vars dans rax
mov     rcx, rax ; Str
call    strlen ; => on met compte le nombre d'élément dans vars
mov     [rbp+var2], eax ; => on met le résultat dans var2
mov     [rbp+i], 0 ; => on met 0 à la variable i
jmp     short loc_40150B

```

On a 4 pointeurs qui pointent sur des adresses.

- **Vars** pointe sur **[rbp-30]** (-48 en décimal) et est de type char (car 1 octet avec byte)
- **Var2** pointe sur **[rbp-8]** (8 en décimal (il n'y a pas le h) et est de type int (dword = 4 octets)
- **i** pointe sur **[rbp-4]** (est type int (dword)) (on sait que c'est un compteur)
- Et enfin **var** pointe sur **[rbp+10]** (+16 en décimal).

Quand un offset est positif, ça veut dire que c'est un argument passé à la fonction.

Donc ici var correspond à notre argument.

```

Adresses hautes
↑
[rbp+0x10] ← var (argument de la fonction)
[rbp+0x08] ← adresse de retour (return address)
[rbp+0x00] ← ancienne valeur de rbp (sauvegardée)
----- ← rbp pointe ici
[rbp-0x04] ← i (compteur de boucle)
[rbp-0x08] ← var2
...
[rbp-0x30] ← vars (1 byte, probablement début d'un buffer)
↓
Adresses basses (la stack grandit vers le bas)

```

Le programme rajoute des éléments à certaines adresses.

De plus il faut faire gaffe à ce genre d'information car dans IDA, vars n'est pas une vraie variable au sens C, c'est un alias pour l'offset -30h.

Donc ida va afficher un truc du style :

```
mov [rbp+vars+18h], 0
```

Mais le processeurs le verra de cette manière :

```
mov [rbp + (-0x30) + 0x18], 0
mov [rbp - 0x18], 0
```

Maintenant si on regarde la deuxième partie de notre fonction FUN :

```

IDA View-A  Hex View-1  Local Types  Imports  Exports
.text:00000000004015A9 jmp     short loc_40150B
.text:00000000004015AB:
.text:00000000004015AB loc_4015AB:
.text:00000000004015AB: mov     eax, [rbp+i] ; CODE XREF: FUN+91i;j ; => on met i dans eax (ici 0)
.text:00000000004015AE cdq     eax
.text:00000000004015B0 movzx   eax, [rbp+rax+vars] ; eax = vars[i]
.text:00000000004015B5 mov     edx, [rbp+var] ; on met notre input dans edx
.text:00000000004015B8 xor     edx, eax ; on xor notre input avec vars
.text:00000000004015BA mov     eax, [rbp+i] ; on met la valeur de i dans eax
.text:00000000004015BD cdq     eax
.text:00000000004015BF mov     [rbp+rax+vars], dl ; on met un byte de notre résultat en xor dans vars[i]
.text:00000000004015C3 mov     eax, [rbp+i] ; on remet eax à zéro
.text:00000000004015C6 cdq     eax
.text:00000000004015C8 movzx   eax, [rbp+rax+vars] ; on met la nouvelle valeur de vars[i] dans eax
.text:00000000004015CD movsx   eax, al ; on met le byte de cette valeur dans eax (encore)
.text:00000000004015D0 mov     ecx, eax ; Character
.text:00000000004015D2 call    putchar ; on affiche ce caractère
.text:00000000004015D7 add     [rbp+i], 1 ; on incrémente le i pour la boucle
.text:00000000004015D8 loc_4015D8:
.text:00000000004015D8: mov     eax, [rbp+i] ; CODE XREF: FUN+59i;j
0000009AB 00000000004015AB: FUN:loc_4015AB (Synchronized with Hex View-1)

```

Ici on peut voir que le programme prend notre input (donc 4 octets) puis le xor avec la clé qu'on a trouvé tout à l'heure.

À la fin on récupère le dernier byte (avec dl).

On a trouvé la clé. Ok c'est super mais on ne sait pas ce que le programme souhaite comme input de base. Donc on pourrait faire une sorte de **brute force**.

Bref revenons à notre programme.

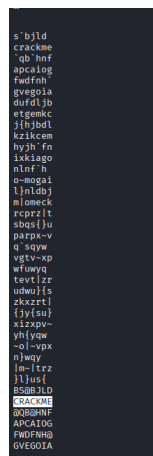
Notre code ressemblerais donc à ça maintenant :

```
#include <stdio.h>
#include <string.h>

int main() {
    unsigned char vars[] = {0x22, 0x33, 0x20, 0x22, 0x2A, 0x2C, 0x24};
    int len = 7;

    for (int key = 0; key < 256; key++) {
        for (int i = 0; i < len; i++) {
            unsigned char c = vars[i] ^ key;
            putchar(c);
        }
        printf("\n");
    }
    return 0;
}
```

Ce qui nous donne :



```
s`bJld
crackme
qb`hnf
apcalog
twdfmh
gvegoia
dufdljb
etgenac
j{hjbdl
kzikcem
hyjh`fn
ixkiago
nlnf`h
u-mugzi
l}nldbj
momeck
reprzlt
sbasllu
parpx-v
q`sqyw
vgtv-wp
wfuwq
tevtlzzr
uduwlls
zkzrtl{
jyj{su}
xizxp~
yh{ydw
-ol-vpx
njwqy
lme-lrz
l}lus{
s`sqwld
s`crackme
s`qb@hnf
apcalog
twdfmh
gvegoia
```

Donc la clé est 65 en décimal et le mot de passe était crackme.

